The Journal of Open Source Software

# asdex: Automatic Sparse Differentiation in JAX

**Adrian Hill** [1,2¶] **and Guillaume Dalle** [3]

**1** BIFOLD – Berlin Institute for the Foundations of Learning and Data, Berlin, Germany **2** Machine Learning Group, Technical University of Berlin, Berlin, Germany **3** LVMT, ENPC, Institut Polytechnique de Paris, Univ Gustave Eiffel, Marne-la-Vallée, France ¶ Corresponding author

**DOI:** 10.xxxxxx/draft

**Software**

- Review
- Repository
- Archive

**Editor:**

**Submitted:** 22 October 2026
**Published:** unpublished

**License**
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

## Summary

Many tasks in scientific computing and machine learning require the Jacobian or Hessian matrix of a function. Automatic differentiation (AD) computes these derivatives to machine precision (Baydin et al., 2018; Blondel & Roulet, 2026; Griewank & Walther, 2008), but materializing a dense $m \times n$ Jacobian requires $n$ forward-mode or $m$ reverse-mode AD passes, one per column or row. For a large class of functions, each output depends on only a few inputs, making the derivative matrix *sparse*. Automatic sparse differentiation (ASD) exploits this structure in four steps (Hill et al., 2025): *detection* of the input-agnostic sparsity pattern, *coloring* of a graph to group columns or rows that can share an AD pass, *compressed differentiation* to compute a compressed derivative matrix with one AD pass per color, and finally *decompression* into the original sparsity pattern. The number of colors, and hence of AD passes, is often independent of the problem dimension: a banded Jacobian with $b$ contiguous bands, for instance, only ever requires $b$ colors, regardless of its size. `asdex`[1] offers the first standalone ASD toolkit in the popular JAX (Bradbury et al., 2018) ecosystem. With `asdex.jacobian` and `asdex.hessian`, it provides sparse drop-in replacements for `jax.jacobian` and `jax.hessian`.

## Statement of need

Sparse derivative matrices arise in nonlinear systems of equations, second-order optimization algorithms, sensitivity analysis, and many other applications. The target audience for `asdex` is researchers and practitioners in scientific machine learning who wish to leverage JAX's performance, JIT compilation, and accelerator support.

JAX's built-in `jacfwd`, `jacrev`, and `hessian` functions materialize *dense* derivative matrices, performing one AD pass per input or output dimension. This standard approach quickly hits two main limitations: a *memory* bottleneck, if the dense matrix is too large to store, and a high *computational* cost, as the number of AD passes scales with the matrix dimension. Computing a sparse Jacobian (Curtis et al., 1974) or Hessian (Coleman & Moré, 1984; Powell & Toint, 1979) with ASD alleviates both of these hurdles by combining more efficient matrix storage with a faster way to fill it. See Gebremedhin et al. (2005) and Griewank & Walther (2008, Chapter 8) for thorough reviews of this topic.

## State of the field

ASD initially matured in low-level programming languages such as Fortran and C++, but we restrict our review to high-level languages, which machine learning research favors for the rapid prototyping workflows they enable.

[1] https://github.com/adrhill/asdex

DRAFT

A general-purpose ASD implementation in a high-level, open-source language recently appeared in Julia (Bezanson et al., 2017), through the combination of `DifferentiationInterface.jl` (Dalle & Hill, 2026), `SparseConnectivityTracer.jl` (Hill & Dalle, 2025) for sparsity pattern detection, and `SparseMatrixColorings.jl` (SMC) (Montoison et al., 2025) for coloring. These packages, to which the present authors contributed, now serve as core sparse differentiation infrastructure for downstream software such as `NonlinearSolve.jl` (Pal et al., 2026). The speedups they enable are documented in Hill & Dalle (2025).

No general-purpose equivalent exists for JAX, although there are a few prototypes. We list them in the table below with their main features:

| Package | Detection | Coloring |
|---|---|---|
| `sparsejac`[2] | None | Distance-1 (from `networkx`) |
| `sparsediffax`[3] | None | Distance-2 & star (from SMC) |
| `jax-nansparse`[4] | `NaN` tracing | None |
| `jax2sympy`[5] | `jaxpr` symbolic conversion | None |
| `asdex` | `jaxpr` tracing | Distance-2 & star (based on SMC) |

A `jaxpr`-based approach to sparsity pattern detection has been outlined by Simpson (2024). Parts of the ASD pipeline have also been reimplemented internally by application-specific libraries. The sparse linear solver `JAX-AMG`[6] (Liu et al., 2026) combines `jaxpr`-based sparsity detection with a parallel greedy distance-1 coloring, while the finite element framework `tatva`[7] (Pundir et al., 2026) pairs `jaxpr`-based detection with greedy distance-2 coloring from its companion package `tatva-coloring`.

We built `asdex` rather than contributing to the general-purpose packages above because none of them combined the "proper" sparsity detection paradigm (`jaxpr` tracing) with the state-of-the-art coloring techniques (distance-2 and star colorings). Additionally, many of the alternatives listed above were made public only recently (we became aware of the concurrent works `JAX-AMG` and `tatva` as we were writing up this paper).

# Software design

## Core features

`asdex` mirrors the main stages of ASD as distinct, composable components.

**Sparsity detection.** `asdex` performs abstract interpretation on a `jaxpr`[8] (short for *JAX expression*), JAX's intermediate representation of a function. Abstract interpretation propagates index sets through each `jaxpr` primitive to determine which inputs influence which outputs, yielding a conservative pattern that may contain false positives but never misses a nonzero. Working at the `jaxpr` level, the detector reuses JAX's own program representation and naturally handles functions built from arbitrary JAX primitives. It also makes second-order detection free: because `jax.grad(f)` produces an ordinary JAX function with a `jaxpr` of its own, and the Hessian of a scalar-valued function `f` is the Jacobian of its gradient, `asdex` obtains Hessian patterns by applying the same first-order detector to `jax.grad(f)`, with no separate second-order detector to implement or maintain.

[2] https://github.com/mfschubert/sparsejac
[3] https://github.com/gdalle/sparsediffax
[4] https://github.com/nardi/jax-nansparse
[5] https://github.com/johnviljoen/jax2sympy
[6] https://github.com/jx-wang-s-group/JAX-AMG
[7] https://github.com/smec-ethz/tatva
[8] https://docs.jax.dev/en/latest/jaxpr.html

DRAFT

**Coloring.** The detected pattern gives rise to a graph coloring problem, which asdex solves approximately using standard greedy algorithms. For Jacobians, a distance-2 coloring of a bipartite row-column graph partitions columns (forward mode) or rows (reverse mode) into structurally orthogonal groups (with no overlapping nonzeros) (Gebremedhin et al., 2005). By default, asdex picks whichever mode requires fewer colors. For Hessians, a star coloring exploits the symmetry of the matrix to further reduce the number of colors (Gebremedhin et al., 2007). In both cases, the resulting groups are encoded in a seed matrix.

**Compressed differentiation and decompression.** The compressed matrix stems from parallelized (`jax.vmap`-ed[9]) Jacobian-vector or vector-Jacobian products, evaluated against the seed matrix. All nonzero coefficients are obtained in this way and then efficiently scattered back into the original sparse format required for the Jacobian or Hessian.

## Performance and interface

**Preparation.** A central design choice separates one-time *preparation* (detection and coloring, which depend on input shapes but not values) from repeated *evaluation* (compressed differentiation and decompression) of the sparse derivative. In an iterative solver or a training loop, preparation is amortized, and only the cheap compressed AD pass is repeated, echoing the preparation mechanism of `DifferentiationInterface.jl` (Dalle & Hill, 2026). While detection and coloring are not JAX transformations themselves, the prepared derivative function they produce is an ordinary JAX function, so it composes with `jax.jit` and `jax.vmap` and runs on CPU, GPU, and TPU backends.

The API deliberately stays close to JAX's own and matches its semantics: where a dense Jacobian is written as `jax.jacobian(f)`, `asdex.jacobian(f, x)` differs in only two aspects:

- it takes a sample input `x` for preparation
- it returns a function computing a sparse matrix (defaulting to JAX's `BCOO` format)

```python
import jax
import asdex

def f(x):
    return (x[1:] - x[:-1]) ** 2

x = jax.numpy.zeros(1000)
# Preparation: detect sparsity & color, return jit-able function
jac_fn = jax.jit(asdex.jacobian(f, x))
# Evaluation: cheap compressed AD passes and decompression
J = jac_fn(x)
```

Beyond this core, asdex mirrors the breadth of JAX's derivative API. Like JAX's own transformations, it accepts arbitrary `PyTree` inputs and outputs rather than only flat vectors. Multiple arguments (with the `argnums` keyword, enabling partial differentiation) and auxiliary outputs (with the `has_aux` keyword) are supported as well. Users can further supply known sparsity patterns, reuse colorings across sessions, limit their memory usage, and decompress results into the dense and sparse formats of their choice, including those from `jax.experimental.sparse`, `numpy`, and `scipy.sparse`. The package is developed openly under the MIT license. Its documentation spans a tutorial, how-to guides, an API reference, a contributor's guide, and continuously published benchmarks tracking detection, coloring, and evaluation performance.

[9] By specifying a `chunk_size` in `asdex`, the number of products computed in parallel can be limited via `jax.lax.map`, bounding memory usage.

DRAFT

## Research impact statement

asdex was inspired by JAX issue #1032[10], a feature request for sparse Jacobian and Hessian support, which has remained open since 2019. This issue centralizes most of the related discussions and shows that there is sustained interest in such a feature in the JAX ecosystem. As for asdex itself, it has already been applied to accelerate the computation of Hessians in machine learning interatomic potentials (Langer et al., 2026) and has been integrated into `splineax`[11], a sparsity-focused extension of the popular linear and least-squares library `lineax` (Rader et al., 2023).

## AI usage disclosure

Generative AI tools (Anthropic's Claude Opus 4.5 to 5.5 and Fable 5 models) were used in developing the asdex software and writing this manuscript. All AI-generated code and text were reviewed and verified by the authors. The design of asdex is based directly on prior work by the authors that was written without generative AI tools, namely the peer-reviewed Julia packages `DifferentiationInterface.jl` (Dalle & Hill, 2026), `SparseConnectivityTracer.jl` (Hill & Dalle, 2025), and `SparseMatrixColorings.jl` (Montoison et al., 2025), from which it inherits its algorithmic approach.

## Acknowledgements

Adrian Hill gratefully acknowledges funding from the German Federal Ministry for Research, Technology and Space under the grant BIFOLD26B. We thank Alexis Montoison for his work on the Julia ASD ecosystem, Marcel Langer and Denis Korolev for their feedback on asdex, as well as Jonathan Brodrick, John Viljoen and Johanna Hafner for productive discussions around JAX.

## References

Baydin, A. G., Pearlmutter, B. A., Radul, A. A., & Siskind, J. M. (2018). Automatic Differentiation in Machine Learning: A Survey. *Journal of Machine Learning Research*, *18*, 153:1–153:43. https://jmlr.org/papers/v18/17-468.html

Bezanson, J., Edelman, A., Karpinski, S., & Shah, V. B. (2017). Julia: A fresh approach to numerical computing. *SIAM Review*, *59*(1), 65–98. https://doi.org/10.1137/141000671

Blondel, M., & Roulet, V. (2026). *The elements of differentiable programming*. Draft version 4, arXiv:2403.14606v4. https://doi.org/10.48550/arXiv.2403.14606

Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Katariya, Y., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., & Zhang, Q. (2018). *JAX: Composable transformations of Python+NumPy programs* (Version 0.3.13). http://github.com/jax-ml/jax

Coleman, T. F., & Moré, J. J. (1984). Estimation of sparse Hessian matrices and graph coloring problems. *Mathematical Programming*, *28*(3), 243–270. https://doi.org/10.1007/BF02612334

Curtis, A. R., Powell, M. J. D., & Reid, J. K. (1974). On the estimation of sparse Jacobian matrices. *IMA Journal of Applied Mathematics*, *13*(1), 117–119. https://doi.org/10.1093/imamat/13.1.117

[10] https://github.com/jax-ml/jax/issues/1032
[11] https://github.com/nardi/splineax

DRAFT

Dalle, G., & Hill, A. (2026). A common interface for automatic differentiation. *Journal of Machine Learning Research*, *27*, 25:1–25:13. https://jmlr.org/papers/v27/25-1024.html

Gebremedhin, A. H., Manne, F., & Pothen, A. (2005). What color is your Jacobian? Graph coloring for computing derivatives. *SIAM Review*, *47*(4), 629–705. https://doi.org/10.1137/s0036144504444711

Gebremedhin, A. H., Tarafdar, A., Manne, F., & Pothen, A. (2007). New acyclic and star coloring algorithms with application to computing Hessians. *SIAM Journal on Scientific Computing*, *29*(3), 1042–1072. https://doi.org/10.1137/050639879

Griewank, A., & Walther, A. (2008). *Evaluating derivatives: Principles and techniques of algorithmic differentiation* (2nd ed.). Society for Industrial and Applied Mathematics. https://doi.org/10.1137/1.9780898717761

Hill, A., & Dalle, G. (2025). Sparser, Better, Faster, Stronger: Sparsity Detection for Efficient Automatic Differentiation. *Transactions on Machine Learning Research*. https://openreview.net/forum?id=GtXSN52nIW

Hill, A., Dalle, G., & Montoison, A. (2025). An illustrated guide to automatic sparse differentiation. *The Fourth Blogpost Track at ICLR 2025*. https://openreview.net/forum?id=ykZibuSbJj

Langer, M. F., Hill, A., & Ceriotti, M. (2026). Truncated automatic sparse differentiation for machine learning interatomic potentials. *NeurIPS 2026 Workshop on Sim2Science: ML with Imperfect Scientific Models*. https://openreview.net/forum?id=Ec1UuQEzG6

Liu, Y., Fan, X., & Wang, J.-X. (2026). JAX-AMG: A GPU-accelerated differentiable sparse linear solver library for JAX. *SoftwareX*, *35*, 102966. https://doi.org/10.1016/j.softx.2026.102966

Montoison, A., Dalle, G., & Gebremedhin, A. H. (2025). *Revisiting sparse matrix coloring and bicoloring*. arXiv:2505.07308. https://doi.org/10.48550/arXiv.2505.07308

Pal, A., Holtorf, F., Larsson, A., Loman, T. E., Utkarsh, U., Schäfer, F., Qu, Q., Edelman, A., & Rackauckas, C. (2026). NonlinearSolve.jl: High-performance and robust solvers for systems of nonlinear equations in Julia. *ACM Transactions on Mathematical Software*, *52*(1), 1:1–1:26. https://doi.org/10.1145/3779117

Powell, M. J. D., & Toint, Ph. L. (1979). On the estimation of sparse Hessian matrices. *SIAM Journal on Numerical Analysis*, *16*(6), 1060–1074. https://doi.org/10.1137/0716078

Pundir, M., Lorez, F., & Kammer, D. S. (2026). *A versatile FEM framework with native GPU scalability via globally-applied AD*. arXiv:2602.12365. https://doi.org/10.48550/arXiv.2602.12365

Rader, J., Lyons, T. J., & Kidger, P. (2023). *Lineax: Unified linear solves and linear least-squares in JAX and Equinox*. arXiv:2311.17283. https://doi.org/10.48550/arXiv.2311.17283

Simpson, D. (2024). *An unexpected detour into partially symbolic, sparsity-expoiting autodiff; or Lord won't you buy me a Laplace approximation*. Blog post, Un garçon pas comme les autres (Bayes). https://dansblog.netlify.app/posts/2024-05-08-laplace/laplace


DRAFT